

Lec01 - Algorithm, RAM Model, Analysis

Algorithm: finite, clear step-by-step procedure to solve a problem. Input -> output. Properties: correct, unambiguous, general, simple, finite/effective. Analysis predicts time + memory, compares algorithms and helps choose efficient solution. Cases: best=min steps, worst=max steps, average=expected steps. RAM model: one processor; assignment, arithmetic, comparison, memory access and print treated as constant-time steps. Exact counting is hard and varies by machine.

Big O / Omega / Theta + Patterns

Big O = upper bound / worst growth. Omega = lower bound / best growth. Theta = tight bound. Ignore constants + lower-order terms. Growth: $\log n < n < n \log n < n^2 < n^3 < 2^n < n!$. Patterns: fixed loop -> $O(1)$; one loop $n \rightarrow O(n)$; separate loops $n+n \rightarrow O(n)$; nested $n*n \rightarrow O(n^2)$; nested $n*fixed \rightarrow O(n)$; $i=i*2$ or $i=i/2 \rightarrow O(\log n)$; outer n and inner $\log n \rightarrow O(n \log n)$; $1+2+...+n = n(n+1)/2 \rightarrow O(n^2)$.

RAM / Runtime Answer Template

Booklet pattern for runtime questions:
1) state loop count. 2) state work per iteration. 3) write $T(n)$. 4) choose biggest term. Example: for $i=1..n$ {sum=sum+A[i]} -> loop n times, each iteration constant -> $T(n)=c1+c2n \rightarrow O(n)$.

Sorting Basics: Insertion, Bubble, Selection

Insertion Sort: builds sorted left part; best sorted $O(n)$, worst reverse $O(n^2)$, space $O(1)$. Bubble Sort: compare adjacent; swap if wrong; largest moves to end each pass. Best $O(n)$ with no-swap flag; avg/worst $O(n^2)$. Selection Sort: select min from unsorted part and swap with first unsorted position. Best/avg/worst $O(n^2)$; swaps about $n-1$. Pseudo: INSERTION key=A[j], shift while $A[i]>key$. BUBBLE if $A[j]>A[j+1]$ swap. SELECTION find min index then swap.

Lec03 - Stack

Stack follows LIFO: last in, first out. Access only TOP. push=add to top; pop=remove/return top; peek=read top only. Empty: top=-1. Full: top==maxSize-1. Trace rule: push increments top then stores value; pop returns current top then decrements. Apps: string/word reversal, undo, browser history, recursion/function calls. Underflow=pop empty; overflow=push full.

Lec04 - Queue + Circular Queue

Queue follows FIFO: first in, first out. insert/enqueue at rear; remove/dequeue from front; peekFront reads front without remove. Empty: count/nItems==0. Full: count==maxSize. Linear queue limitation: removed front spaces can be wasted. Circular queue uses wrap-around: rear=(rear+1)%maxSize, front=(front+1)%maxSize. If no count and one empty slot: full when (rear+1)%size==front. Trace logical queue from front for count items, not raw array order.

Lec05 - Linked Lists

Linked list: nodes connected by links. Node = data + next. Last node next=NULL. Sequential access, dynamic size, non-contiguous memory. Array: direct index access, fixed size, contiguous memory. Operations: find, insert, delete. Double-ended list has first + last. Doubly linked list has next + previous and traverses forward/backward. Applications: stacks/queues, browser/music next-prev, adjacency lists, dynamic memory.

Linked List Pointer Rules

Pointer rules:
Insert first: new.next=first; first=new.
Insert after current: new.next=current.next; current.next=new.
Delete first: first=first.next.
Delete middle/last: previous.next=current.next.
Traverse all nodes: current=first; while(current!=null){... current=current.next;}
Trap: if condition is current.next!=null, the last node will NOT be processed.

Lec06 - Tree + BST

Tree: nodes + edges. Root=top. Parent has children. Leaf=no children. Binary tree=max 2 children. Path=connected nodes; subtree=node with descendants. Key value used for search. BST: left key < parent, right key >= parent. Insert/search: start from root; smaller -> left; greater/equal -> right. In insertion sequence, first value becomes root unless a diagram gives root. Min=go left repeatedly. Max=go right repeatedly.

Traversals + Binary Tree Terms

BFS/Level order: level by level. DFS traversals:
Inorder LNR = Left, Node, Right. In BST gives ascending order.
Preorder NLR = Node, Left, Right.
Postorder LRN = Left, Right, Node.
Degree=number of children. Depth=edges from root to node. Height=max depth. Full binary tree: every node has 0 or 2 children; nodes= $2^{(h+1)}-1$. Complete binary tree: last level filled left to right; height=floor(log2(n)).

BST Delete + Successor

BST delete cases:
1) Leaf/no child -> remove parent link.
2) One child -> parent links directly to child.
3) Two children -> replace with inorder successor or predecessor, then delete successor from old position.
Inorder successor = smallest value in right subtree: go to right child once, then go left until no left child. Always keep BST order.

BST Quick Example

BST quick example - insert 50,30,70,20,40:

```
graph TD
    50 --> 30
    50 --> 70
    30 --> 20
    30 --> 40
```

Inorder: 20,30,40,50,70
Preorder: 50,30,20,40,70
Postorder: 20,40,30,70,50

Student ID :
Module Code : IT1170

Lec08 - Recursion + Recurrences

Recursion: function calls itself to solve a smaller version. Must have base case/final case. factorial(0)=1; factorial(n)=n*factorial(n-1). factorial(4)=4*3*2*1=24. power(x,0)=1; power(x,n)=x*power(x,n-1). Recurrence equation expresses time using smaller input. Factorial: $T(n)=T(n-1)+O(1) \rightarrow O(n)$. Binary search: $T(n)=T(n/2)+O(1) \rightarrow O(\log n)$.

Master Method

Master Method for $T(n)=aT(n/b)+f(n)$. a =#subproblems, b =size reduction factor, $f(n)$ =divide/combine cost. Compare $f(n)$ with $n^{(\log_b a)}$. If $f(n)$ smaller -> $\Theta(n^{(\log_b a)})$. If equal -> $\Theta(n^{(\log_b a)} \log n)$. If larger and regularity holds -> $\Theta(f(n))$. Merge/Quick best: $a=2, b=2, f(n)=n \rightarrow \Theta(n \log n)$.

Lec09 - Divide and Conquer + Quick Sort

Divide and conquer: divide problem, conquer subproblems, combine solutions. Quick Sort: choose pivot; partition so smaller values left and greater values right; recursively quicksort left and right. PARTITION last pivot: $x=A[r]$, $i=p-1$, scan $j=p..r-1$; if $A[j]<=x$ then $i++$ and swap $A[i],A[j]$; finally swap $A[i+1],A[r]$. Best/average balanced: $T(n)=2T(n/2)+\Theta(n)=\Theta(n \log n)$. Worst unbalanced: $T(n)=T(n-1)+\Theta(n)=\Theta(n^2)$.

Lec10 - Merge Sort

Merge Sort: divide array into two halves, recursively sort, then merge. Better worst case than insertion sort. Recurrence: $T(n)=2T(n/2)+\Theta(n)$. Complexity: $O(n \log n)$. Merge uses temporary L/R arrays and sentinels or index checks. Same order of growth for best/average/worst because it divides and merges similarly regardless of initial order.

Lec11-12 - Heap + Heapsort

Heap: array represented as complete binary tree. Max heap: parent >= children. Min heap: parent <= children. 1-based array formulas: $PARENT(i)=\text{floor}(i/2)$, $LEFT(i)=2i$, $RIGHT(i)=2i+1$. Root $A[1]$. MAX_HEAPIFY fixes violation at subtree root; $O(\log n)$. BUILD_MAX_HEAP starts from last non-leaf node floor(n/2) downto 1; $O(n)$. Heapsort: build max heap, swap root with last, reduce heap size, heapify root; $O(n \log n)$. Priority queue: INSERT, EXTRACT_MAX; both $O(\log n)$.

Heap / Trace Rules

Heap trace rules:
BUILD_MAX_HEAP starts $i=\text{floor}(n/2)$.
After heapsort swap: reduce heap_size, MAX_HEAPIFY(A,1).
Max heap root = largest. For $n=10$, start index=floor(10/2)=5. Parent of $i=6$ is floor(6/2)=3. Trace by comparing parent with left/right and swapping with the largest.

StackX Java Template

```
class StackX{
    int maxSize, top; int[] st;
    StackX(int s){maxSize=s; st=new int[s]; top=-1;}
    boolean isEmpty(){return top==-1;}
    boolean isFull(){return top==maxSize-1;}
    void push(int x){ if(isFull()) print("Full"); else st[++top]=x; }
    int pop(){ if(isEmpty()){print("Empty"); return -99;} return st[top--];
    }
    int peek(){ if(isEmpty()) return -99; return st[top]; }
    int getCount(){return top+1;}
}
```

Stack Trace + Applications

Frame trace: write index/values; top arrow points to last valid item. push(x): top increases then x stored. pop(): current top returned/removed then top decreases. peek(): no change.

Search stack: pop to temp stack while comparing, then restore by popping temp back.

Reverse string/word: push characters, then pop until empty.

QueueArray Java Template

```
class QueueArray{
    int maxSize, front, rear, count; int[] q;
    QueueArray(int s){maxSize=s; q=new int[s]; front=0; rear=-1; count=0;}
    boolean isEmpty(){return count==0;}
    boolean isFull(){return rear==maxSize-1;} // linear
    void insert(int x){ if(isFull()) print("Full"); else{q[++rear]=x; count++;} }
    int remove(){ if(isEmpty()) return -1; int t=q[front++]; count--; return t; }
    int peekFront(){ if(isEmpty()) return -1; return q[front]; }
}
```

CircularQueue Java Template

```
class CircularQueue{
    int maxSize, front, rear, count; int[] q;
    CircularQueue(int s){maxSize=s; q=new int[s]; front=0; rear=-1; count=0
    ;}
    boolean isEmpty(){return count==0;}
    boolean isFull(){return count==maxSize;}
    void insert(int x){ if(isFull()) print("Full"); else{ rear=(rear+1)%max
    Size; q[rear]=x; count++; } }
    int remove(){ if(isEmpty()) return -1; int t=q[front]; front=(front+1)%
    maxSize; count--; return t; }
    int peekFront(){ if(isEmpty()) return -1; return q[front]; }
    void display(){int i=front; for(int c=0;c<count;c++){print(q[i]); i=(i+
    1)%maxSize;}}
}
```

Link + LinkList Key Methods

```
class Link{int id,marks; Link next; Link(int i,int m){id=i;marks=m;next=null
;}}
class LinkList{
    Link first=null; boolean isEmpty(){return first==null;}
    void insertFirst(int id,int marks){Link n=new Link(id,marks); n.next=first;
    first=n;}
    Link find(int key){Link cur=first; while(cur!=null){if(cur.id==key)return c
ur; cur=cur.next;} return null;}
    boolean insertAfter(int key,int id,int marks){Link cur=first; while(cur!=nu
ll && cur.id!=key) cur=cur.next; if(cur==null)return false; Link n=new Lin
k(id,marks); n.next=cur.next; cur.next=n; return true;}
    Link deleteFirst(){if(first==null)return null; Link t=first; first=first.ne
xt; return t;}
    boolean delete(int key){Link cur=first, prev=null; while(cur!=null && cur.i
d!=key){prev=cur; cur=cur.next;} if(cur==null)return false; if(cur==first)
    first=first.next; else prev.next=cur.next; return true;}
}
```

Common Scenario Methods + Booklet Templates

Common scenario methods:

count vacant beds: count=0; cur=first; while(cur!=null){ if(cur.vacant==1) count++; cur=cur.next;} return count.

assign first vacant: bed=findVacant(); if(bed!=null){bed.assignBed(); print(bed.bedNo);} else print("Patient beds are not available").

displayGreaterThanRoot(): inorder(root.right).

Booklet templates: runtime -> loop count -> total -> biggest term -> Big O. BST -> first value root; smaller left; greater/equal right. Heapify -> compare parent,left,right; swap with largest and repeat.

BST Node + Tree Insert/Find

```
class Node{int key; String name; Node left,right; Node(int k,String n){key=k
;name=n;}}
class Tree{ Node root=null;
    void insert(int key,String name){Node n=new Node(key,name); if(root==null){
    root=n;return;} Node cur=root,parent; while(true){parent=cur; if(key<cur.k
ey){cur=cur.left; if(cur==null){parent.left=n;return;}} else{cur=cur.right
; if(cur==null){parent.right=n;return;}}}}
    Node find(int key){Node cur=root; while(cur!=null && cur.key!=key){ if(key<
cur.key) cur=cur.left; else cur=cur.right;} return cur;}
    Node minimum(){Node cur=root; if(cur==null)return null; while(cur.left!=nul
l) cur=cur.left; return cur;}
    Node maximum(){Node cur=root; if(cur==null)return null; while(cur.right!=nu
ll) cur=cur.right; return cur;}
}
```

BST Traversal + Delete Mini Logic

```
void inOrder(Node r){ if(r!=null){ inOrder(r.left); display(r); inOrder(r.righ
t); } }
void preOrder(Node r){ if(r!=null){ display(r); preOrder(r.left); preOrder(r
.right); } }
void postOrder(Node r){ if(r!=null){ postOrder(r.left); postOrder(r.right);
display(r); } }
Tree delete mini logic: leaf -> parent link=null. one child -> parent link=c
hild. two children -> successor=minimum node in right subtree; copy succes
sor data; delete successor from old location. Complete tree height=floor(l
og2(n)); full tree nodes=2^(h+1)-1.
```

Student ID :
Module Code : IT1170

Recursion Code Patterns

```
int fact(int n){ if(n==0) return 1; return n*fact(n-1); }
int power(int x,int n){ if(n==0) return 1; return x*power(x,n-1); }
int sum(int n){ if(n==0) return 0; return n+sum(n-1); }
Always identify: base case + recursive call + smaller input.
```

Quick Sort / Partition